

LINGUAGEM DE PROGRAMAÇÃO II

Funções, Métodos e Recursividade



Curso: Ciência da Computação
Universidade Estadual da Paraíba (UEPB)

O que vamos aprender hoje?

- 1 Por que criar **funções e métodos**
- 2 Como **declarar** e **chamar** um método
- 3 Parâmetros, argumentos e valores de **retorno**
- 4 **Escopo**: onde cada variável "existe"
- 5 **Sobrecarga**: o mesmo nome, comportamentos diferentes
- 6 **Recursividade**: uma função que chama a si mesma

Como ainda não vimos objetos, todo método aqui será `static`, chamado direto do `main` – exatamente como já fazemos.

- 1 Por que Funções e Métodos?
- 2 Declarando e Chamando Métodos
- 3 Escopo de Variáveis
- 4 Sobrecarga de Métodos
- 5 Recursividade

- 1 Por que Funções e Métodos?
- 2 Declarando e Chamando Métodos
- 3 Escopo de Variáveis
- 4 Sobrecarga de Métodos
- 5 Recursividade

Um Problema Comum

- Imagine precisar calcular a média de notas em **vários** lugares do programa;
- Copiar e colar o mesmo cálculo várias vezes deixa o código enorme e difícil de corrigir;
- Se um dia o cálculo mudar, você precisaria achar **todas** as cópias.

A Solução: Reutilizar Código

😊 Analogia

Pense numa **receita de bolo** guardada num caderno. Toda vez que você quer fazer o bolo, não reescreve a receita – só **consulta** ela de novo. Uma função é exatamente isso: um pedaço de código guardado, pronto para ser usado quantas vezes quisermos.

Função x Método

- Em muitas linguagens, blocos de código reutilizáveis são chamados de **funções**;
- Em Java, todo código mora **dentro** de uma classe – por isso, o nome oficial é **método**;
- No dia a dia, os dois termos acabam sendo usados quase como sinônimos.

Lembra do main?

- `public static void main(String[] args)` é, ele mesmo, um **método**;
- A JVM simplesmente **chama** o método `main` para começar a executar o programa.

Procedimento x Função

- **Procedimento:** um método que executa uma ação, mas **não devolve** nenhum valor (void);
- **Função:** um método que **devolve** um valor, usando return.

Vamos ver os dois tipos com calma daqui a pouco.

♡ Checkpoint

- Funções/métodos evitam repetir código;
- Em Java, chamamos de **método** (mora dentro de uma classe);
- `main` também é um método;
- **Procedimento** não devolve valor; **função** devolve, com `return`.

Sumário

- 1 Por que Funções e Métodos?
- 2 Declarando e Chamando Métodos
- 3 Escopo de Variáveis
- 4 Sobrecarga de Métodos
- 5 Recursividade

Anatomia de um Método

```
1 public static void nomeDoMetodo() {  
2     // código do método  
3 }
```

Vamos quebrar essa linha em pedaços que já conhecemos.

Pedacço 1: public static

```
1 public static void nomeDoMetodo() {
```

- **public**: o método pode ser chamado de qualquer lugar do programa;
- **static**: assim como o `main`, o método pertence à classe – por isso conseguimos chamá-lo direto, sem criar nenhum objeto.

Dica

Como ainda não estudamos objetos, **todo** método desta aula será **static** – igual ao `main` que já conhecemos.

Pedaco 2: retorno, nome e parênteses

```
public static void nomeDoMetodo() {
```

- `void`: por enquanto, indica que o método **não devolve** nada (um procedimento);
- `nomeDoMetodo`: o nome que damos ao método;
- `()`: os parênteses guardam os **parâmetros** – por enquanto, vazios.

😊 Exemplo: Um metodo bem simples

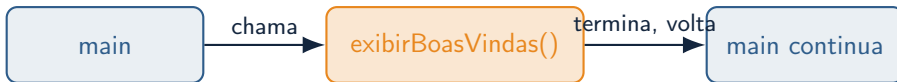
```
1 public static void exibirBoasVindas() {  
2     System.out.println("Bem vindo ao sistema!");  
3 }
```

Chamando o Método

```
1 public static void exibirBoasVindas() {  
2     System.out.println("Bem vindo ao sistema!");  
3 }  
4  
5 public static void main(String[] args) {  
6     exibirBoasVindas();  
7 }
```

Para **executar** um método, escrevemos seu nome seguido de `()` – isso se chama **chamar** o método.

Visualizando a Chamada



Quando o método termina, a execução **volta** para logo depois de onde foi chamado.

- E se quisermos que `exibirBoasVindas` cumprimente uma pessoa **específica**?
- Precisamos passar essa informação para dentro do método – usando **parâmetros**.

Sintaxe de Parâmetros

```
1 public static void saudar(String nome) {  
2     System.out.println("Ola, " + nome + "!");  
3 }
```

- String nome: um **parâmetro** – uma variável que só existe dentro do método;
- O valor de nome vem de fora, no momento da chamada.

Exemplo Prático

😊 Exemplo: Chamando com parametro

```
1 saudar ("Maria");  
2 saudar ("Joao");
```

Cada chamada usa um valor diferente – o mesmo método, reaproveitado.

Parâmetro x Argumento

Termo	O que é
Parâmetro	a variável na declaração do método
Argumento	o valor enviado na chamada

Em `saudar(String nome)`, `nome` é o parâmetro. Em `saudar("Maria")`, `"Maria"` é o argumento.

Múltiplos Parâmetros

```
1 public static void somar(int a, int b) {  
2     System.out.println("Soma: " + (a + b));  
3 }
```

- Separamos vários parâmetros por **vírgula**;
- A ordem importa: o primeiro argumento vai para a, o segundo para b.

😊 Exemplo: Chamando com dois argumentos

```
1 somar(3, 5); // Soma: 8
```

- O `somar` anterior só **imprime** o resultado;
- Mas e se quisermos **usar** esse resultado depois, em outro cálculo?
- Para isso, o método precisa **devolver** um valor.

A Palavra-chave return

```
1 public static int somar(int a, int b) {  
2     return a + b;  
3 }
```

- Trocamos void pelo **tipo** do valor devolvido (int, aqui);
- return a + b; devolve o resultado e **encerra** o método imediatamente.

Usando o Valor Retornado

```
1 int resultado = somar(3, 5);  
2 System.out.println(resultado); // 8
```

- O retorno pode ser guardado numa variável;
- Ou usado **direto**: `System.out.println(somar(3, 5));`

void x Retorno

	Procedimento	Função
Palavra no lugar do tipo	void	tipo do valor (int, ...)
Usa return com valor?	Não	Sim
Pode ser usado num println?	Não	Sim

Exemplo Prático

☺ Exemplo: Verificando se e par

```
1 public static boolean ehPar(int numero) {  
2     return numero % 2 == 0;  
3 }
```

```
1 if (ehPar(8)) {  
2     System.out.println("E par!");  
3 }
```

Repare: uma função pode devolver qualquer tipo, inclusive boolean.

😊 Exemplo: O maior entre dois numeros

```
1 public static int maior(int a, int b) {  
2     if (a > b) {  
3         return a;  
4     }  
5     return b;  
6 }
```

Erro Comum

Declarar um método com um tipo de retorno (`int`, `boolean`, ...) e **esquecer** o `return`.
O Java nem deixa compilar: "missing return statement".

♡ Checkpoint

- `public static`: por enquanto, sempre presentes – `static` porque ainda não usamos objetos;
- Parâmetro (na declaração) x argumento (na chamada);
- `void`: procedimento, não devolve nada;
- Tipo de retorno + `return`: função, devolve um valor.

- 1 Por que Funções e Métodos?
- 2 Declarando e Chamando Métodos
- 3 Escopo de Variáveis**
- 4 Sobrecarga de Métodos
- 5 Recursividade

Onde uma Variável "Existe"?

😊 Definição: Escopo

É a região do código onde uma variável **existe** e pode ser usada. Uma variável declarada **dentro** de um método só existe **dentro** dele.

Escopo: uma analogia

😊 Analogia

Pense em cada método como um **cômodo** de uma casa. O que você deixa em cima da mesa do seu quarto não aparece magicamente na cozinha. Cada cômodo (método) tem suas próprias coisas (variáveis).

Exemplo: Fora de Escopo

```
1 public static void metodo1() {  
2     int x = 10;  
3 }  
4  
5 public static void main(String[] args) {  
6     metodo1();  
7     System.out.println(x); // erro de compilacao!  
8 }
```

☹ Erro Comum

A variável **x** foi criada **dentro** de `metodo1` – assim que o método termina, ela deixa de existir. O `main` nunca teve acesso a ela.

Parâmetros também são Locais

- Um parâmetro é, na prática, uma variável local do método;
- Em `saudar(String nome)`, a variável `nome` só existe **dentro** de `saudar`.

♥ Checkpoint

- Variáveis declaradas dentro de um método só existem **ali dentro**;
- Isso vale também para os parâmetros;
- Cada método tem seu próprio "espaço" de variáveis, isolado dos outros.

- 1 Por que Funções e Métodos?
- 2 Declarando e Chamando Métodos
- 3 Escopo de Variáveis
- 4 Sobrecarga de Métodos**
- 5 Recursividade

Mesmo Nome, Situações Diferentes

- Às vezes queremos somar dois `int`, mas também dois `double`;
- Ter que inventar nomes diferentes (`somarInt`, `somarDouble`) é chato de lembrar.

O que é Sobrecarga?

😊 Definição: Sobrecarga (*overloading*)

É quando **dois ou mais métodos** têm o **mesmo nome**, mas parâmetros **diferentes** (em quantidade ou tipo). O Java decide qual usar de acordo com os argumentos da chamada.

Exemplo Prático

😊 Exemplo: Somando inteiros ou decimais

```
1 public static int somar(int a, int b) {  
2     return a + b;  
3 }  
4  
5 public static double somar(double a, double b) {  
6     return a + b;  
7 }
```

Como o Java Escolhe

```
1 somar(2, 3);           // chama a versao com int  
2 somar(2.5, 3.1);      // chama a versao com double
```

O Java olha os **tipos dos argumentos** na chamada e escolhe automaticamente o método certo.

☹ Erro Comum

Achar que dá para sobrecarregar mudando **só** o nome dos parâmetros ou **só** o tipo de retorno. O Java exige que a **lista de parâmetros** (quantidade e/ou tipos) seja diferente.

♡ Checkpoint

- Sobrecarga: mesmo nome, parâmetros diferentes;
- O Java escolhe a versão certa pelos argumentos da chamada;
- Só mudar o retorno (mantendo os mesmos parâmetros) **não** é sobrecarga válida.

- 1 Por que Funções e Métodos?
- 2 Declarando e Chamando Métodos
- 3 Escopo de Variáveis
- 4 Sobrecarga de Métodos
- 5 Recursividade**

Um Jeito Diferente de Repetir

- Já sabemos repetir tarefas com `for` e `while`;
- Existe **outra** forma, muito poderosa: fazer um método chamar **a si mesmo**.

O que é Recursão?

😊 Definição: Recursão

É quando uma função (ou método) chama a **si mesma**, para resolver um problema **menor** do mesmo tipo, até chegar num caso simples o bastante para responder direto.

Recursão: uma analogia

😊 Analogia

Pense numas **bonecas russas** (matrioscas): dentro de cada boneca tem outra, um pouco menor, até chegar na **menorzinha**, que não abre mais. Cada chamada recursiva é como abrir uma boneca – até chegar na menor, o **caso base**.

As Duas Partes de uma Recursão

- ❶ **Caso base:** a situação mais simples, que já sabemos responder **sem** chamar a função de novo;
- ❷ **Caso recursivo:** chama a própria função, com um problema **menor** (mais perto do caso base).

☹ **Erro Comum**

Sem um caso base que realmente seja alcançado, a recursão **nunca para** – e o programa quebra com `StackOverflowError`.

Exemplo 1: Somar de 1 até N

- **Caso base:** somar até 0 dá 0;
- **Caso recursivo:** somar até n é n **mais** a soma até $n - 1$.

Exemplo 1: o Código

😊 Exemplo: somaAte

```
1 public static int somaAte(int n) {  
2     if (n == 0) {  
3         return 0;  
4     }  
5     return n + somaAte(n - 1);  
6 }
```

Exemplo 1: Desenrolando

$$\begin{aligned}\text{somaAte}(4) &= 4 + \text{somaAte}(3) \\ &= 4 + (3 + \text{somaAte}(2)) \\ &= 4 + (3 + (2 + \text{somaAte}(1))) \\ &= 4 + (3 + (2 + (1 + \text{somaAte}(0)))) \\ &= 4 + (3 + (2 + (1 + 0))) = 10\end{aligned}$$

Exemplo 2: Fatorial

- O fatorial de n (escrito $n!$) é o produto de todos os inteiros de 1 até n ;
- **Caso base:** $0! = 1$ (por definição);
- **Caso recursivo:** $n! = n \times (n-1)!$

Exemplo 2: o Código

😊 Exemplo: fatorial

```
1 public static int fatorial(int n) {  
2     if (n == 0) {  
3         return 1;  
4     }  
5     return n * fatorial(n - 1);  
6 }
```

Exemplo 2: Desenrolando

$$\begin{aligned}\text{fatorial}(4) &= 4 \times \text{fatorial}(3) \\ &= 4 \times (3 \times \text{fatorial}(2)) \\ &= 4 \times (3 \times (2 \times \text{fatorial}(1))) \\ &= 4 \times (3 \times (2 \times (1 \times \text{fatorial}(0)))) \\ &= 4 \times (3 \times (2 \times (1 \times 1))) = 24\end{aligned}$$

Exemplo 3: Fibonacci

- A sequência de Fibonacci: 0, 1, 1, 2, 3, 5, 8, ... – cada número é a soma dos dois anteriores;
- **Casos base:** $\text{fibonacci}(0) = 0$ e $\text{fibonacci}(1) = 1$;
- **Caso recursivo:** $\text{fibonacci}(n) = \text{fibonacci}(n-1) + \text{fibonacci}(n-2)$.

Exemplo 3: o Código

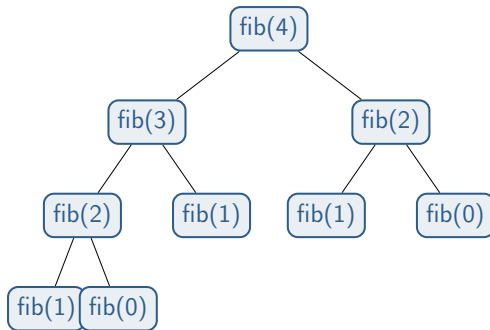
☺ Exemplo: fibonacci

```
1 public static int fibonacci(int n) {  
2     if (n == 0) {  
3         return 0;  
4     }  
5     if (n == 1) {  
6         return 1;  
7     }  
8     return fibonacci(n - 1) + fibonacci(n - 2);  
9 }
```

Exemplo 3: Uma Recursão Diferente

- Repare: fibonacci chama a si mesma **duas** vezes, não uma;
- Isso faz a quantidade de chamadas **crescer bastante** conforme n aumenta.

Árvore de Chamadas: fibonacci(4)



Note que `fib(2)` e `fib(1)` são calculados **mais de uma vez!**

Lembra da aula de Strings?

- O método `inverterRecursivo(String s)` também era uma recursão;
- Caso base: `s.isEmpty()`; caso recursivo: inverter o restante e colocar a primeira letra no final.

Recursão x Iteração

Recursão	Iteração (for/while)
Método chama a si mesmo	Usa um laço de repetição
Precisa de um caso base	Precisa de uma condição de parada
Costuma usar mais memória	Geralmente mais econômica
Às vezes deixa o código mais claro	Às vezes é mais direto

♡ Checkpoint

- Recursão: a função chama a si mesma, com um problema menor;
- Sempre precisa de um caso base – senão, `StackOverflowError`;
- `fibonacci` mostra que uma função pode se chamar mais de uma vez por chamada;
- Tudo que é feito com recursão também pode ser feito com `for/while`, e vice-versa.

- ➊ Métodos evitam repetir código – `main` já é um exemplo de método;
- ➋ **Procedimento** (`void`) não devolve valor; **função** devolve, com `return`;
- ➌ Parâmetro é declarado no método; argumento é o valor passado na chamada;
- ➍ Variáveis (e parâmetros) só existem dentro do método onde foram criadas – isso é **escopo**;
- ➎ **Sobrecarga**: mesmo nome, parâmetros diferentes;
- ➏ **Recursão**: um método chama a si mesmo, sempre com um caso base para parar.

Mãos à Obra!